Human Verification: `Containers.Cofunctor`

Morphisms of Directed Containers Are Cofunctors: **DCont** $\cong$ **Cof**

Machine-generated from `lean/Cofunctor.lean` for review by Neil Ghani

July 2026

Abstract

This document presents every definition and theorem *statement* from the Lean 4 module `Containers.Cofunctor` in three parallel forms: the original Lean, a readable Haskell rendering, and precise mathematical English. Proofs are omitted — they are machine-checked by Lean. The purpose is to let a human verifier confirm that the formalisation captures the intended mathematics.

Contents

1 Overview **2**

2 Dictionary vocabulary **2**
2.1 Definition: `DirectedContainer.idHom` . . . 2
2.2 Definition: `DirectedContainer.cod` . . . 3
2.3 Definition: `DirectedContainer.after` . . . 3

3 Morphisms of directed containers **3**
3.1 Structure: `DContMorphism` . . . 3

4 Cofunctors **4**
4.1 Structure: `Cofunctor` . . . 4

5 The bijection DCont$(C, D) \cong$ Cof(C, D) **5**
5.1 Definition: `DContMorphism.toCofunctor` . . . 5
5.2 Definition: `Cofunctor.toDContMorphism` . . . 5
5.3 Theorem: `toCofunctor_toDContMorphism` . . . 5
5.4 Theorem: `toDContMorphism_toCofunctor` . . . 6

6 Dependent bookkeeping for composition **6**
6.1 Theorem: `Cofunctor.comp_transport` . . . 6

7 The category Cof of cofunctors **6**
7.1 Definition: `Cofunctor.id` . . . 6
7.2 Definition: `Cofunctor.comp` . . . 7
7.3 Theorem: `Cofunctor.id_comp` . . . 7
7.4 Theorem: `Cofunctor.comp_id` . . . 8
7.5 Theorem: `Cofunctor.comp_assoc` . . . 8

8 Identity and composition for DCont **8**
8.1 Definition: `DContMorphism.id` . . . 8
8.2 Definition: `DContMorphism.comp` . . . 9

9 Functoriality of the isomorphism Φ **9**
9.1 Theorem: `DContMorphism.toCofunctor_id` 9
9.2 Theorem: `DContMorphism.toCofunctor_comp` 9

10 Translation conventions **10**

1 Overview

This file formalises the *morphism level* of the directed-container/small- category dictionary (Ahman–Chapman–Uustalu; Shapiro–Spivak): the morphisms native to directed containers are **cofunctors** (also called *retrofunctors*), not ordinary functors.

The dictionary. Every directed container C presents a small category $\mathrm{cat}(C)$:

- objects are shapes $C.\mathrm{Shape}$;
- a morphism out of s is a position $p : P(s)$, with codomain $s \downarrow p$;
- the identity at s is the root $o(s)$;
- composition $q \circ p$ is the shift $p \oplus q$.

The variance point. A container morphism $(f, f^\sharp)$ carries $f : \mathrm{Shape}_C \to \mathrm{Shape}_D$ *forwards* but $f^\sharp_s : P_D(f(s)) \to P_C(s)$ *backwards*. A functor would need $P_C(s) \to P_D(f(s))$ — the wrong variance. With compatibility laws M0/M1/M2, the pair $(f, f^\sharp)$ is precisely a **cofunctor** (C0/C1/C2) of the presented categories.

Main results.

1. **DContMorphism**$(C, D) \cong$ **Cofunctor**(C, D) as sets — the two structures carry identical data; the round-trips are definitional equalities.
2. Cofunctors form a *strict* category **Cof** (laws hold by `rfl`).
3. The assignment $\Phi = (\cdot).\mathrm{toCofunctor}$ preserves identities and composition, hence is an isomorphism of categories **DCont** $\cong$ **Cof**.

2 Dictionary vocabulary

These three definitions name the data of the presented category in terms of the directed-container fields, enabling the statement of the cofunctor laws without directly mentioning `root`/`sub`/`shift`.

2.1 Definition: `DirectedContainer.idHom`

Definition 2.1 (Identity morphism in the presented category)**.** The **identity** at s in $\mathrm{cat}(C)$ is the root position $o(s)$.

$$\mathrm{idHom}(C)(s) \;:=\; C.\mathrm{root}(s) \;\in\; P(s).$$

Lean.

```
def DirectedContainer.idHom (C : DirectedContainer) (s : C.Shape) :
    C.Pos s := C.root s
```

Haskell.
```
idHom :: DirectedContainer s p ⇒ s → p
idHom s = root s
```

2.2 Definition: `DirectedContainer.cod`

Definition 2.2 (Codomain of a morphism)**.** The **codomain** of a morphism $p : P(s)$ (out of s) is the sub-shape $s \downarrow p$:

$$\mathrm{cod}(C)(s)(p) \;:=\; C.\mathrm{sub}(s)(p).$$

Lean.
```
def DirectedContainer.cod (C : DirectedContainer) (s : C.Shape)
    (p : C.Pos s) : C.Shape := C.sub s p
```

Haskell.
```
cod :: DirectedContainer s p ⇒ s → p → s
cod s p = sub s p
```

2.3 Definition: `DirectedContainer.after`

Definition 2.3 (Composition in the presented category)**.** The **composite** $q \circ p$ ("q after p"), where p is a morphism out of s and q is a morphism out of $s \downarrow p$, is the shift $p \oplus q$:

$$\mathrm{after}(C)(s)(p)(q) \;:=\; C.\mathrm{shift}(s)(p)(q) \;\in\; P(s).$$

Lean.
```
def DirectedContainer.after (C : DirectedContainer) (s : C.Shape)
    (p : C.Pos s) (q : C.Pos (C.sub s p)) : C.Pos s := C.shift s p q
```

Haskell.
```
after :: DirectedContainer s p ⇒ s → p → p → p
after s p q = shift s p q
```

3 Morphisms of directed containers

3.1 Structure: `DContMorphism`

Definition 3.1 (Morphism of directed containers)**.** A **directed-container morphism** $C \to D$ is a pair $(f, f^\sharp)$ with:

- $f : \mathrm{Shape}_C \to \mathrm{Shape}_D$ — a forward shape map;
- $f^\sharp_s : P_D(f(s)) \to P_C(s)$ for each s — a contravariant position map;

satisfying:

M0. $f(s \downarrow f^\sharp_s(h)) = f(s) \downarrow_D h$ (preserves codomains over f);

M1. $f^\sharp_s(o_D(f(s))) = o_C(s)$ (preserves roots);

M2. $f^\sharp_s(h \oplus_D k) = f^\sharp_s(h) \oplus_C f^\sharp_{s \downarrow f^\sharp_s(h)}(\mathrm{M0}^{-1} \triangleright k)$ (compatible with shifts, with transport along M0).

Lean.
```
structure DContMorphism (C D : DirectedContainer) where
  f      : C.Shape → D.Shape
  fSharp : (s : C.Shape) → D.Pos (f s) → C.Pos s
  m0 : ∀ (s : C.Shape) (h : D.Pos (f s)),
    f (C.sub s (fSharp s h)) = D.sub (f s) h
```

```
  m1 : ∀ (s : C.Shape),
    fSharp s (D.root (f s)) = C.root s
  m2 : ∀ (s : C.Shape) (h : D.Pos (f s))
              (k : D.Pos (D.sub (f s) h)),
    fSharp s (D.shift (f s) h k)
      = C.shift s (fSharp s h)
          (fSharp (C.sub s (fSharp s h)) ((m0 s h).symm |→ k))
```

Haskell.

```
data DContMorph c d = DContMorph
  { dF      :: c → d
  , dFSharp :: c → d → c    -- fSharp s h : C.Pos s  (backward)
  -- Laws (as QuickCheck properties):
  -- M0: dF (sub c s (dFSharp m s h)) == sub d (dF m s) h
  -- M1: dFSharp m s (root d (dF m s)) == root c s
  -- M2: dFSharp m s (shift d (dF m s) h k)
  --       == shift c s (dFSharp m s h) (dFSharp m (sub c s (dFSharp m
     s h)) k)
  --     (transport invisible in Haskell)
  }
```

Fidelity note. Law M2 in Lean carries the transport $(\mathrm{M0}^{-1} \triangleright k)$, which coerces $k : P_D(f(s) \downarrow_D h)$ to $P_D(f(s \downarrow_C f_s^\sharp(h)))$ (these types differ because M0 is not definitional). In Haskell, positions are a single type, so this transport is invisible and M2 simply asserts that $f^\sharp$ is a homomorphism for shift.

4 Cofunctors

4.1 Structure: `Cofunctor`

Definition 4.1 (Cofunctor)**.** A **cofunctor** $\varphi : \mathrm{cat}(C) \to \mathrm{cat}(D)$ consists of:

- an object map $\mathrm{obj} : \mathrm{Shape}_C \to \mathrm{Shape}_D$;
- a *lift*: for each s and each morphism h out of $\mathrm{obj}(s)$ in $\mathrm{cat}(D)$, a morphism $\mathrm{lift}_s(h)$ out of s in $\mathrm{cat}(C)$;

satisfying:

C0. $\mathrm{obj}(\mathrm{cod}_C(s, \mathrm{lift}_s(h))) = \mathrm{cod}_D(\mathrm{obj}(s), h)$ (lift preserves codomains);

C1. $\mathrm{lift}_s(\mathrm{idHom}_D(\mathrm{obj}(s))) = \mathrm{idHom}_C(s)$ (lift sends identities to identities);

C2. $\mathrm{lift}_s(\mathrm{after}_D(\mathrm{obj}(s), h, k)) = \mathrm{after}_C(s, \mathrm{lift}_s(h), \mathrm{lift}_{\mathrm{cod}_C(s,\mathrm{lift}_s(h))}(\mathrm{C0}^{-1} \triangleright k))$ (lift is compatible with composition).

Lean.

```
structure Cofunctor (C D : DirectedContainer) where
  obj  : C.Shape → D.Shape
  lift : (s : C.Shape) → D.Pos (obj s) → C.Pos s
  c0 : ∀ (s : C.Shape) (h : D.Pos (obj s)),
    obj (C.cod s (lift s h)) = D.cod (obj s) h
  c1 : ∀ (s : C.Shape),
    lift s (D.idHom (obj s)) = C.idHom s
  c2 : ∀ (s : C.Shape) (h : D.Pos (obj s))
              (k : D.Pos (D.cod (obj s) h)),
    lift s (D.after (obj s) h k)
      = C.after s (lift s h)
          (lift (C.cod s (lift s h)) ((c0 s h).symm |→ k))
```

Haskell.

```haskell
data Cofunctor c d = Cofunctor
  { obj  :: c → d
  , lift :: c → d → c    -- lift s h : Pos_C s (backward map)
  -- Laws:
  -- C0: obj (cod c s (lift phi s h)) == cod d (obj phi s) h
  -- C1: lift phi s (idHom d (obj phi s)) == idHom c s
  -- C2: lift phi s (after d (obj phi s) h k)
  --       == after c s (lift phi s h) (lift phi (cod c s (lift phi s
     h)) k)
  }
```

5 The bijection $\mathbf{DCont}(C, D) \cong \mathbf{Cof}(C, D)$

5.1 Definition: `DContMorphism.toCofunctor`

Definition 5.1 (Forward direction of the bijection)**.** Every directed-container morphism $(f, f^\sharp)$ yields a cofunctor: set obj $= f$, lift $= f^\sharp$, and carry the laws M0/M1/M2 verbatim as C0/C1/C2.

Lean.

```
def DContMorphism.toCofunctor {C D : DirectedContainer}
    (m : DContMorphism C D) : Cofunctor C D :=
  ⟨m.f, m.fSharp, m.m0, m.m1, m.m2⟩
```

Haskell.

```haskell
toCofunctor :: DContMorph c d → Cofunctor c d
toCofunctor m = Cofunctor (dF m) (dFSharp m)
-- Laws M0/M1/M2 become C0/C1/C2 directly.
```

5.2 Definition: `Cofunctor.toDContMorphism`

Definition 5.2 (Backward direction of the bijection)**.** Every cofunctor (obj, lift) yields a directed-container morphism: set $f =$ obj, $f^\sharp =$ lift, and carry C0/C1/C2 verbatim as M0/M1/M2.

Lean.

```
def Cofunctor.toDContMorphism {C D : DirectedContainer}
    (phi : Cofunctor C D) : DContMorphism C D :=
  ⟨phi.obj, phi.lift, phi.c0, phi.c1, phi.c2⟩
```

Haskell.

```haskell
toDContMorphism :: Cofunctor c d → DContMorph c d
toDContMorphism phi = DContMorph (obj phi) (lift phi)
```

5.3 Theorem: `toCofunctor_toDContMorphism`

Theorem 5.3 (Round-trip, morphism direction)**.** The composite toCofunctor∘toDContMorphism is the identity on $\mathbf{DContMorphism}(C, D)$:

$$m.\text{toCofunctor.toDContMorphism} \;=\; m.$$

Holds definitionally (`rfl`) — the two conversions are transparent record re-packagings.

Lean.

```
theorem toCofunctor_toDContMorphism {C D : DirectedContainer}
    (m : DContMorphism C D) :
    m.toCofunctor.toDContMorphism = m
```

Haskell.
```
-- toDContMorphism (toCofunctor m) = m
-- (trivially: both are record re-packagings of the same fields)
```

5.4 Theorem: `toDContMorphism_toCofunctor`

Theorem 5.4 (Round-trip, cofunctor direction)**.** The composite toDContMorphism∘toCofunctor is the identity on $\mathbf{Cofunctor}(C, D)$:

$$\varphi.\text{toDContMorphism.toCofunctor} \;=\; \varphi.$$

Together with the previous theorem this establishes the hom-set bijection $\mathbf{DCont}(C, D) \cong \mathbf{Cof}(C, D)$. Both directions hold definitionally.

Lean.
```
theorem toDContMorphism_toCofunctor {C D : DirectedContainer}
    (phi : Cofunctor C D) :
    phi.toDContMorphism.toCofunctor = phi
```

Haskell.
```
-- toCofunctor (toDContMorphism phi) = phi
```

6 Dependent bookkeeping for composition

6.1 Theorem: `Cofunctor.comp_transport`

Theorem 6.1 (Transport commutation for composite lifts)**.** Let $\psi : \text{cat}(D) \to \text{cat}(E)$ be a cofunctor, let $e : y = x$ in D.Shape, and let $e_2 : \psi.\text{obj}(x) = z$. Then:

$$e^{-1} \triangleright (\psi.\text{lift}(x, e_2^{-1} \triangleright k)) \;=\; \psi.\text{lift}(y,\ (e_2 \circ \text{cong}(\psi.\text{obj}, e))^{-1} \triangleright k).$$

This is the transport identity that makes the composite shift law C2 typecheck in the definition of `Cofunctor.comp`.

Lean.
```
theorem Cofunctor.comp_transport {D E : DirectedContainer}
    (psi : Cofunctor D E)
    {x y : D.Shape} (e : y = x) {z : E.Shape} (e2 : psi.obj x = z)
    (k : E.Pos z) :
    e.symm ▸ (psi.lift x (e2.symm ▸ k))
      = psi.lift y (((congrArg psi.obj e).trans e2).symm ▸ k)
```

Haskell.
```
-- This is pure dependent bookkeeping with no Haskell analogue.
-- In Haskell all types are non-dependent so transports are invisible.
```

Fidelity note. This lemma is the sole piece of genuinely dependent proof infrastructure unique to the composition of cofunctors. It packages the transport that arises when the codomain law C0 for the composite involves two chained equalities. In Haskell, composition of the lift maps is simply $\text{lift}_\varphi(s, \text{lift}_\psi(\varphi(s), m))$ with no transport, because all shape equalities are invisible.

7 The category Cof of cofunctors

7.1 Definition: `Cofunctor.id`

Definition 7.1 (Identity cofunctor)**.** The **identity cofunctor** on $\text{cat}(C)$ has object map id and lift id. All three laws hold definitionally.

Lean.

```
def Cofunctor.id (C : DirectedContainer) : Cofunctor C C where
  obj  := fun s ⇒ s
  lift := fun _ p ⇒ p
  c0   := fun _ _ ⇒ rfl
  c1   := fun _ ⇒ rfl
  c2   := fun _ _ _ ⇒ rfl
```

Haskell.

```
idCofunctor :: Cofunctor c c
idCofunctor = Cofunctor id (\_ p → p)
```

7.2 Definition: `Cofunctor.comp`

Definition 7.2 (Composition of cofunctors)**.** Given cofunctors $\varphi : \mathrm{cat}(C) \to \mathrm{cat}(D)$ and $\psi : \mathrm{cat}(D) \to \mathrm{cat}(E)$, their **composite** $\psi \circ \varphi : \mathrm{cat}(C) \to \mathrm{cat}(E)$ has:

$$(\psi \circ \varphi).\mathrm{obj} \;=\; \psi.\mathrm{obj} \circ \varphi.\mathrm{obj},$$

$$(\psi \circ \varphi).\mathrm{lift}(s, m) \;=\; \varphi.\mathrm{lift}(s,\; \psi.\mathrm{lift}(\varphi.\mathrm{obj}(s), m)).$$

Object maps compose forwards; lifts compose *backwards*. The composite codomain law chains C0 for φ and ψ; the composite shift law requires `Cofunctor.comp_transport`.

Lean.

```
def Cofunctor.comp {C D E : DirectedContainer}
    (phi : Cofunctor C D) (psi : Cofunctor D E) : Cofunctor C E where
  obj  := fun s ⇒ psi.obj (phi.obj s)
  lift := fun s m ⇒ phi.lift s (psi.lift (phi.obj s) m)
  c0   := fun s h ⇒
    (congrArg psi.obj (phi.c0 s (psi.lift (phi.obj s) h))).trans
      (psi.c0 (phi.obj s) h)
  c1   := fun s ⇒
    (congrArg (phi.lift s) (psi.c1 (phi.obj s))).trans (phi.c1 s)
  c2   := by
    intro s h k
    rw [psi.c2 (phi.obj s) h k, phi.c2 s (psi.lift (phi.obj s) h) _,
      Cofunctor.comp_transport psi (phi.c0 s (psi.lift (phi.obj s) h))
        (psi.c0 (phi.obj s) h) k]
```

Haskell.

```
compCofunctor :: Cofunctor c d → Cofunctor d e → Cofunctor c e
compCofunctor phi psi = Cofunctor
  { obj  = obj psi . obj phi
  , lift = \s m → lift phi s (lift psi (obj phi s) m)
  }
-- Laws for the composite follow from those of phi and psi.
```

7.3 Theorem: `Cofunctor.id_comp`

Theorem 7.3 (Left unit law for **Cof**)**.** For every cofunctor $\varphi : \mathrm{cat}(C) \to \mathrm{cat}(D)$,

$$\mathrm{id}_C \circ \varphi \;=\; \varphi.$$

Holds definitionally (`rfl`).

Lean.

```
theorem Cofunctor.id_comp {C D : DirectedContainer}
    (phi : Cofunctor C D) :
    (Cofunctor.id C).comp phi = phi
```

Haskell.
```
-- compCofunctor idCofunctor phi = phi
```

7.4 Theorem: `Cofunctor.comp_id`

Theorem 7.4 (Right unit law for **Cof**)**.** For every cofunctor $\varphi : \mathrm{cat}(C) \to \mathrm{cat}(D)$,

$$\varphi \circ \mathrm{id}_D \;=\; \varphi.$$

Holds definitionally (`rfl`).

Lean.
```
theorem Cofunctor.comp_id {C D : DirectedContainer}
    (phi : Cofunctor C D) :
    phi.comp (Cofunctor.id D) = phi
```

Haskell.
```
-- compCofunctor phi idCofunctor = phi
```

7.5 Theorem: `Cofunctor.comp_assoc`

Theorem 7.5 (Associativity in **Cof**)**.** For every composable triple φ, ψ, χ,

$$(\chi \circ \psi) \circ \varphi \;=\; \chi \circ (\psi \circ \varphi).$$

Holds definitionally (`rfl`) — **Cof** is a *strict* category.

Lean.
```
theorem Cofunctor.comp_assoc {C D E F : DirectedContainer}
    (phi : Cofunctor C D) (psi : Cofunctor D E) (chi : Cofunctor E F)
       :
    (phi.comp psi).comp chi = phi.comp (psi.comp chi)
```

Haskell.
```
-- compCofunctor (compCofunctor phi psi) chi
--   = compCofunctor phi (compCofunctor psi chi)
```

8 Identity and composition for DCont

These definitions transport identity and composition from **Cof** to **DCont** via the bijection, confirming that **DCont** itself is also a category.

8.1 Definition: `DContMorphism.id`

Definition 8.1 (Identity directed-container morphism)**.** The identity morphism of directed containers at C, obtained by transporting `Cofunctor.id` along the bijection.

Lean.
```
def DContMorphism.id (C : DirectedContainer) : DContMorphism C C :=
  (Cofunctor.id C).toDContMorphism
```

Haskell.
```
idDCont :: DContMorph c c
idDCont = toDContMorphism idCofunctor
```

8.2 Definition: `DContMorphism.comp`

Definition 8.2 (Composition of directed-container morphisms)**.** Composition in **DCont**, obtained by transporting `Cofunctor.comp` along the bijection.

Lean.
```
def DContMorphism.comp {C D E : DirectedContainer}
    (m : DContMorphism C D) (n : DContMorphism D E) : DContMorphism C
       E :=
  (m.toCofunctor.comp n.toCofunctor).toDContMorphism
```

Haskell.
```
compDCont :: DContMorph c d → DContMorph d e → DContMorph c e
compDCont m n =
  toDContMorphism (compCofunctor (toCofunctor m) (toCofunctor n))
```

9 Functoriality of the isomorphism Φ

These two theorems show that the assignment $\Phi = (\cdot).\mathrm{toCofunctor}$ — which is already a bijection on hom-sets and identity on objects — also preserves identities and composition. Together they make Φ an isomorphism of categories $\mathbf{DCont} \cong \mathbf{Cof}$.

9.1 Theorem: `DContMorphism.toCofunctor_id`

Theorem 9.1 (Φ preserves identities)**.**

$$\Phi(\mathrm{id}_C^{\mathbf{DCont}}) \;=\; \mathrm{id}_C^{\mathbf{Cof}}.$$

Holds definitionally (`rfl`).

Lean.
```
theorem DContMorphism.toCofunctor_id (C : DirectedContainer) :
    (DContMorphism.id C).toCofunctor = Cofunctor.id C
```

Haskell.
```
-- toCofunctor idDCont = idCofunctor
```

9.2 Theorem: `DContMorphism.toCofunctor_comp`

Theorem 9.2 (Φ preserves composition)**.** For every composable pair $m : C \to D$, $n : D \to E$,

$$\Phi(n \circ m) \;=\; \Phi(n) \circ \Phi(m).$$

Holds definitionally (`rfl`). Being identity-on-objects, bijective on every hom-set, and functorial, Φ is an isomorphism of categories $\mathbf{DCont} \cong \mathbf{Cof}$.

Lean.
```
theorem DContMorphism.toCofunctor_comp {C D E : DirectedContainer}
    (m : DContMorphism C D) (n : DContMorphism D E) :
    (m.comp n).toCofunctor = m.toCofunctor.comp n.toCofunctor
```

Haskell.
```
-- toCofunctor (compDCont m n)
--   = compCofunctor (toCofunctor m) (toCofunctor n)
```

10 Translation conventions

1. **Cofunctors vs. functors.** A cofunctor $\varphi : \mathcal{C} \to \mathcal{D}$ maps objects in the same direction as a functor, but maps morphisms in the *opposite* direction: it lifts a morphism out of $\varphi(s)$ in $\mathcal{D}$ to a morphism out of s in $\mathcal{C}$. This backward morphism map corresponds directly to the contravariant position map $f_s^\sharp : P_D(f(s)) \to P_C(s)$ of a container morphism.

2. **Definitional strictness.** All category laws for **Cof** (`id_comp`, `comp_id`, `comp_assoc`) and the hom-set bijection (`toCofunctor_toDContMorphism`, `toDContMorphism_toCofunctor`) hold by `rfl`. This reflects the fact that M0/M1/M2 and C0/C1/C2 are *the same axioms* under a mere renaming, so the conversion functions are transparent record projections.

3. **Transport in M2/C2.** The shift compatibility law M2 (and its cofunctor counterpart C2) carries a transport ($\mathrm{M0}^{-1} \triangleright k$), exactly as D5 transports along D4 in `Containers.Directed`. The helper `Cofunctor.comp_transport` packages the additional transport arising in the composite shift law.

4. **Isomorphism of categories.** The final result is an isomorphism **DCont** $\cong$ **Cof**, established component by component: identity on objects, bijective on hom-sets (by the round-trip theorems), and functorial (by `toCofunctor_id` and `toCofunctor_comp`).

5. **Haskell fidelity.** The Haskell renderings lose the dependent typing (all positions are a single type parameter) and hence all transports. The categorical content — that DCont morphisms are cofunctors, that cofunctors form a category, and that the bijection is functorial — is fully visible.